

Systèmes Intelligents avancés Maxime Ambard

A. Etude d'un LSTM pour la prédiction de séries temporelles

Cette partie est adaptée de la ressource suivante:

<https://www.crosstab.io/articles/time-series-pytorch-lstm/>

Nous allons dans cette première partie créer un réseau LSTM pour prédire la concentration de particules fines (PM2.5) à Austin, connaissant les concentrations mesurées dans 5 autres villes du Texas (Del Rio, McAllen, Midlothian, Midland et Houston) sur les instants précédents. Les mesures sont espacées de 2 minutes. Nous avons donc affaire à un modèle many-to-one où plusieurs données d'entrée correspondant à plusieurs timestamps sont utilisées pour prédire la sortie à un seul timestamp.

Le fichier de données `processed_pm25.csv` vous est fourni avec ce sujet (il n'est plus téléchargeable depuis la ressource d'origine).

Packages nécessaires :

torch
pandas
numpy
matplotlib

A1. Création du dataset et preprocessing

La première grande partie de l'application d'un modèle sur des données consiste à mettre les données en forme pour que le modèle puisse s'en servir.

Pour cela, nous allons procéder en 5 étapes :

- Prise de connaissance des données
 - Création de la variable cible et prétraitement (découpage train/test, normalisation)
 - Création et test d'un dataset
 - Création et test d'un dataloader
 - Implémentation en conditions réelles
-
- 1) Créez un script Python permettant de charger le fichier csv "`processed_pm25.csv`" avec pandas, en utilisant la colonne des dates comme index : `pd.read_csv("processed_pm25.csv", index_col="created_at")`. Faites-en une sortie console. Vous pouvez aussi lire le fichier texte brut avec un éditeur de textes. Faites un affichage des données avec Pyplot. Quelques pics très élevés écrasent le graphique : refaites un affichage en limitant l'axe des ordonnées à `[0, 60]` (`plt.ylim`) pour mieux voir les corrélations entre villes.
 - 2) Nous devons fixer quelques paramètres. La ville cible est `target_sensor = "Austin"` et les features sont les 5 autres colonnes : `features = list(df.columns.difference([target_sensor]))`. La colonne Austin ne fait donc pas partie des entrées du modèle. `forecast_lead = 15` sera le décalage temporel entre le dernier instant de la séquence d'input et la valeur prédite (15 pas de 2 minutes, soit 30 minutes). Créez une nouvelle colonne cible `target = f"{target_sensor}_lead{forecast_lead}"` contenant la colonne Austin décalée de `forecast_lead` lignes vers le haut (`shift(-forecast_lead)`), puis supprimez les `forecast_lead` dernières lignes du dataframe, qui ne contiennent plus de valeur

cible. Le décalage temporel est ainsi entièrement géré ici : le Dataset de la question 4 n'aura pas à s'en occuper.

- 3) Séparez le dataframe en 2 parties, l'une pour le train et l'autre pour le test. Pour une série temporelle, on ne mélange pas les données : on coupe à une date. Prenez `test_start = "2021-10-10"`, puis `df_train = df[df.index < test_start].copy()` et `df_test = df[df.index >= test_start].copy()`. Affichez la proportion de données de test. Standardisez ensuite chaque colonne de `df_train` (soustraction de la moyenne et division par l'écart type) et standardisez également les colonnes de `df_test` avec la moyenne et l'écart type calculés sur `df_train`. Avant la standardisation, gardez en mémoire la moyenne et l'écart type de la colonne cible (`target_mean`, `target_stdev`) : ils serviront à dénormaliser les prédictions en A4.
- 4) Créez une classe héritant de `Dataset (torch.utils.data.Dataset)` dont :
 - le constructeur prend en paramètres le tableau de données pandas, le nom de la colonne servant de valeur cible (une seule colonne), la liste des noms des colonnes servant de valeurs de support (features) et la longueur de séquence `sequence_length`. Le constructeur enregistre toutes ces valeurs dans des attributs et transforme le tableau de données pandas en deux tenseurs torch de type float : `X`, de dimension `[N, nombre de features]`, contenant les valeurs d'input, et `y`, de dimension `[N]`, contenant les valeurs de sortie.
 - Surchargez aussi la méthode `__len__`, qui ne prend pas de paramètre et qui renvoie le nombre d'éléments du dataset. On construit une séquence par ligne du dataframe : `__len__` renvoie donc `N`, ce qui permettra en A4 d'avoir exactement une prédiction par ligne.
 - Surchargez la méthode `__getitem__`, qui prend en paramètre un indice `i` et renvoie le couple (séquence d'input, valeur cible). La séquence d'input est formée des lignes `i - sequence_length + 1` à `i` de `X` (dimension `[sequence_length, nombre de features]`) et la cible est `y[i]`, qui correspond déjà à l'instant `t + forecast_lead` grâce au décalage de la question 2. Pour les premiers indices (`i < sequence_length - 1`), il n'y a pas assez de lignes antérieures : complétez le début de la séquence en répétant la première ligne `X[0]` (voir `repeat` et `torch.cat`).
 - Testez le Dataset avec `sequence_length = 4` : affichez `train_dataset[27]` et `train_dataset[28]` et vérifiez que la séquence correspond bien à `df_train[features].iloc[24:28]`. Affichez aussi `train_dataset[0]` pour vérifier le remplissage.
- 5) Création et test des Dataloader
Créez un Dataloader (`torch.utils.data.DataLoader`) prenant en entrée le dataset de la question précédente, avec `batch_size=3` et `shuffle=True`. Récupérez le premier batch avec `next(iter(train_loader))` et affichez sa dimension : vous devez obtenir `[3, 4, 5]`, soit [taille du batch, longueur de séquence, nombre de features].
- 6) Implémentation en conditions réelles
Fixez la graine aléatoire (`torch.manual_seed(101)`), puis prenez `batch_size = 4` et `sequence_length = 30` (1 heure de mesures). Créez `train_dataset` à partir de `df_train` et `test_dataset` à partir de `df_test`, puis les Dataloader correspondants : `train_loader` avec `shuffle=True` et `test_loader` avec `shuffle=False` (on conserve l'ordre chronologique pour les

prédictions). Vérifiez que les dimensions d'un batch sont [4, 30, 5] pour les features et [4] pour la cible.

A2. Création du modèle et de la méthode d'apprentissage

1) Modèle

Créez une classe héritant de `nn.Module` dont le constructeur prend en paramètres le nombre de features d'input, la taille des couches cachées du LSTM et le nombre de features d'output (ici 1). Enregistrez aussi un attribut `num_layers = 1`, le nombre d'étages du LSTM (voir plus bas).

Dans ce constructeur, créez un bloc `nn.LSTM` prenant ces paramètres en configuration (`input_size`, `hidden_size`, `num_layers`). Vous ajouterez le mode `batch_first=True` pour coller au format [B, séquence, features] fourni par le Dataloader.

Toujours dans le constructeur, ajoutez une couche `nn.Linear` prenant pour nombre de features d'entrée la taille de la sortie du LSTM (la taille des couches cachées) et pour taille de sortie le nombre de features de sortie du réseau.

Dans la méthode `forward`, nous devons créer les vecteurs correspondant à `h` et `c` du timestamp 0 (voir le cours). De base, nous pourrions penser que la dimension de ces vecteurs serait égale à [B, H] où B est la taille du batch et H la taille des couches cachées, mais il faut savoir que les modèles LSTM peuvent se construire à plusieurs étages qui s'empilent : des `stacked LSTM`. Dans le module `torch`, la couche LSTM est adaptée à cette potentielle architecture et les vecteurs `h` et `c` ont donc un format [L, B, H] où L est le nombre d'étages LSTM. Dans cette partie, L vaut 1 : les vecteurs que nous devons créer sont donc de dimension [1, B, H]. Utilisez tout de même l'attribut `num_layers` plutôt que la valeur 1 écrite en dur, cela facilitera la partie B1.

Dans la méthode `forward` donc, commencez par créer les vecteurs `h0` et `c0` remplis de zéros (`torch.zeros`), en utilisant le vecteur d'input `x` pour connaître la dimension B (`x.shape[0]`). Donnez en entrée du bloc LSTM l'input et ces deux vecteurs : `_, (hn, _) = self.lstm(x, (h0, c0))`. Le vecteur caché `hn` a lui aussi la dimension [L, B, H] : récupérez celui du dernier étage, `hn[-1]`, de dimension [B, H], et passez-le à la couche linéaire. Vous donnerez en sortie la sortie de la couche linéaire mise en une dimension (`flatten()`), de dimension [B], identique à celle de `y`.

2) Hyperparamètres d'apprentissage

Créez un modèle de la classe ci-dessus avec 5 features d'entrée (`len(features)`), une taille de couches cachées `num_hidden_units = 16` et 1 feature de sortie. Prenez une loss function `MSELoss` et un optimizer de type `Adam` (`learning_rate = 5e-5`). Le nombre d'epochs sera `num_epochs = 2`.

A3. Apprentissage

Faites une méthode `train_model` prenant en entrée un `dataloader`, un modèle, une `loss_function` et un optimizer. Après avoir mis le modèle en mode apprentissage (`model.train()`), faites une boucle sur tous les batches que peut proposer le `DataLoader` et, pour chaque batch, dans cet ordre : un calcul de `forward`, un calcul de `loss`, une suppression des gradients antérieurs (`optimizer.zero_grad()`), un calcul de gradient (`loss.backward()`) et une modification des paramètres du modèle (`optimizer.step()`). Attention : si les gradients sont supprimés après `backward()`, le modèle n'apprend rien. Accumulez la `loss` de chaque batch (`loss.item()`) et affichez la `loss` moyenne en fin d'epoch.

Faites une seconde méthode `test_model` prenant en entrée un dataloader, un modèle et une `loss_function` et faisant le calcul de test du réseau sur tous les batches du dataloader, sans apprentissage : mettez le modèle en mode évaluation (`model.eval()`) et désactivez le calcul des gradients (with `torch.no_grad():`). Affichez la loss moyenne sur les batches.

Faites à la racine du script un premier appel à `test_model` avant tout apprentissage : cette loss du modèle non entraîné servira de point de comparaison. Faites ensuite une boucle qui effectue successivement un appel à la méthode d'apprentissage (sur `train_loader`) et à la méthode de test (sur `test_loader`) pendant `num_epochs` epochs.

A4. Test de prédictions

Refaites un dataloader `train_eval_loader` à partir de `train_dataset` avec un mode shuffle à False, pour obtenir les prédictions dans l'ordre chronologique. Faites une méthode `predict_LSTM` prenant en paramètres un dataloader et le modèle. Dans cette méthode (en mode évaluation et sans calcul de gradient), construisez progressivement un tenseur contenant toutes les sorties calculées par le modèle pour toutes les entrées du dataloader (partez de `torch.tensor([])` et utilisez `torch.cat`). Appliquez-la à `train_eval_loader` et à `test_loader`.

Une fois les données récoltées, repassez en format numpy (`.numpy()`) et ajoutez les prédictions dans une nouvelle colonne de `df_train` et de `df_test` (il y a une prédiction par ligne, puisque `__len__` renvoie N). Dénormalisez ensuite la colonne des prédictions et la colonne cible réelle avec `target_mean` et `target_stddev` enregistrés en A1.3 ($x \times \text{écart type} + \text{moyenne}$).

Faites une méthode permettant d'afficher avec matplotlib les prédictions ainsi que les données réelles sur l'ensemble `df_train` et l'ensemble `df_test` mis bout à bout (`pd.concat`). Vous pouvez utiliser le numéro de ligne comme abscisse et marquer le début du jeu de test par une ligne verticale en `len(df_train)` (`plt.axvline`). Comparez la qualité des prédictions sur le train et sur le test.

B. Essais sur d'autres bases de données et autres hyperparamètres.

B1. Faites la même chose en prenant un stacked LSTM de 3 étages. Ajoutez pour cela un paramètre `num_layers` au constructeur du modèle et passez-le au bloc `nn.LSTM`. Les vecteurs `h0` et `c0` sont alors de dimension `[3, B, H]`, et `hn[-1]` correspond bien à la sortie du dernier étage. Comparez les loss obtenues avec celles du LSTM à un étage (vous pourrez augmenter le nombre d'epochs).

B2. Étudiez l'influence des autres hyperparamètres : longueur de séquence (`sequence_length`), horizon de prédiction (`forecast_lead`), taille des couches cachées, learning rate et nombre d'epochs.

B3. Testez avec d'autres séries temporelles : commencez par prendre une autre ville comme cible, puis essayez un autre jeu de données de votre choix (par exemple une série météorologique ou de consommation électrique).